

Computer Graphics

Contents

1	Computer Graphics	3
1.1	The rendering pipeline	3
1.1.1	Object transformation	3
1.1.2	Perspective projection	4
1.1.3	Clipping	5
1.1.4	Backface culling	5
1.1.5	Rasterisation	6
1.1.6	Hidden surface removal (HSR) and shading	7
1.2	GPU memory architecture	8
1.2.1	Managing time: double buffering	8
1.2.2	Memory cost	9
1.3	The modern programmable pipeline	9
1.3.1	Vertex processing	10
1.3.2	Rasterisation	10
1.3.3	Fragment processing	10
1.4	The polygon model	10
1.4.1	Triangle reduction	11
1.5	Transformations	11
1.6	Fundamental 2D transformations	12
1.6.1	Translation (moving)	12
1.6.2	Rotation (turning)	12
1.6.3	Scaling (resizing)	13
1.6.4	Transformations as matrices	14
1.6.5	Combining transformations	15
1.7	3D transformations	15
1.7.1	The model matrix	16
1.7.2	The view matrix	17
1.7.3	The projection matrix	18
1.7.4	MVP matrix	19
1.7.5	Procedural terrain & object placement	20
1.8	Animation	21
1.8.1	Transformation consistency	21
1.9	Lighting and shading	22
1.9.1	Appearance and materiality	22
1.9.2	HSV/HSL colour space	23
1.9.3	Lighting	23

1.10	Geometry & Texture mapping	27
1.10.1	Vertex attributes	27
1.10.2	Index list	28
1.10.3	Texture mapping	28
1.11	Scene graphs	33
1.11.1	Transformations in a scene graph	34
1.11.2	Attatching a camera	35
1.11.3	CPU vs GPU	35
1.11.4	Three.js example	35
1.12	Ray tracing	36
1.12.1	Rasterisation vs Ray Tracing	36
1.12.2	Ray equation	36
1.12.3	Final colouring	37

1 Computer Graphics

Computer graphics is the field dedicated to generating, manipulating, and synthesising visual content with computers.

The primary goal is to transform an abstract description of a scene (shapes, materials, lights, camera) into a compelling 2D image.

1.1 The rendering pipeline

The rendering pipeline describes how a 3D scene is converted into a 2D image on the screen. These stages reflect the hardware-level process used by the GPU.

The stages are:

1. **Object transformation** - converts object coordinates from local space to world space
2. **Clipping** - removes geometry outside the view
3. **Backface culling** - discards polygons facing away from the camera
4. **Rasterisation** - determines which screen pixels are covered by triangles, producing fragments (potential pixels)
5. **Hidden surface removal (HSR) & Shading** - depth testing to keep only the closest fragment per pixel and computation of final colour

1.1.1 Object transformation

Object transformation is the process of taking a model defined in its own coordinate system (local space) and placing it correctly into the scene (world space).

In the local space, each object is defined relative to its own centre (origin).

- This essentially provides a template or blueprint for the object, which can then be reused easily

In the global space, all objects are placed into a shared global coordinate system, so positions are relative to the world origin.

- Multiple copies of the same object can be placed in different locations, and objects can interact with other objects in the scene

The relation between the local and global space is given by:

$$P_{world} = M \times P_{local}$$

where:

- P_{local} is the vertex in object (local) space
- M is the transformation matrix (model matrix)
- P_{world} is the vertex in world space

M is a 4×4 matrix which combines translation, rotation and scaling. The position of each vertex is calculated independently, using SIMD (Single Instruction, Multiple Data) on the GPU to transform thousands of vertices in parallel (at the same time).

Vertices are represented in homogeneous coordinates as (x, y, z, w) where $w = 1$ for positions, allowing translation to be included in matrix multiplication, while direction vectors use $w = 0$ so

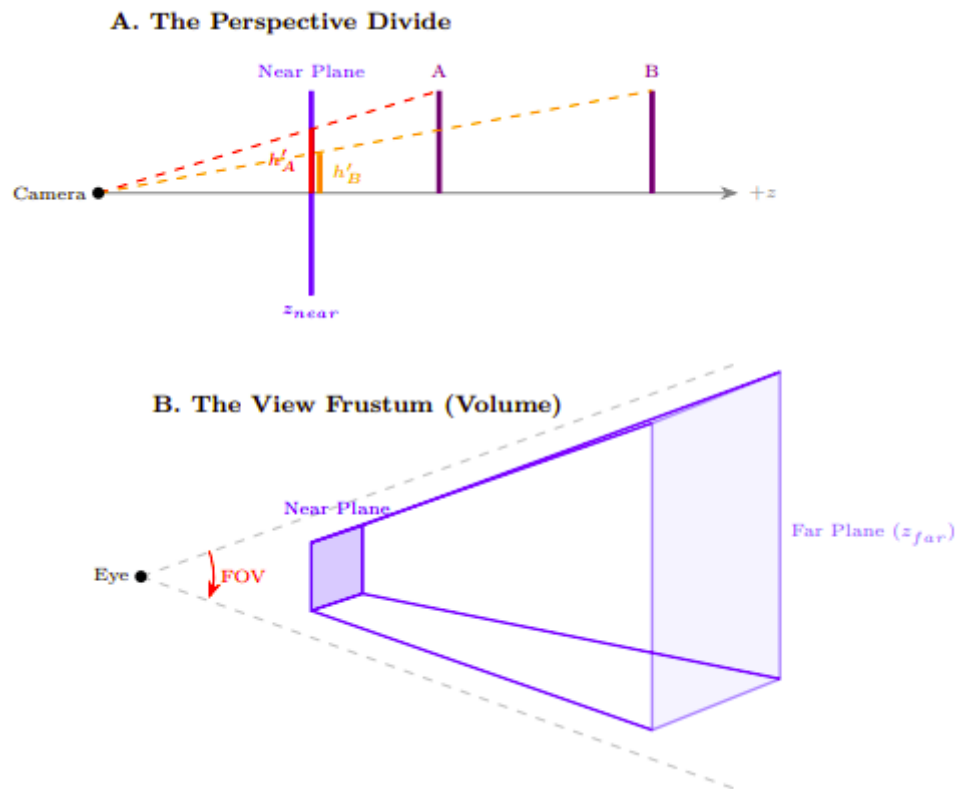
they are unaffected by translation. After applying the projection matrix, w encodes depth-related information of the vertex.

1.1.2 Perspective projection

Perspective projection is the process of simulating a camera lens by transforming 3D coordinates into a 2D representation, where distant objects appear smaller.

To render a 3D scene:

- a **virtual camera** is defined, which establishes the point from which the scene is being viewed
- this camera defines a specific volume of space called the **view frustum**
 - the view frustum is a truncated pyramid that determines exactly what is visible
 - only objects that fall inside the view frustum are rendered
 - the view frustum is limited by:
 - * Field of View (FOV)
 - * the Near Plane (closest visible distance)
 - * the Far Plane (draw distance)
- a **projection matrix** transforms coordinates into **clip space**, mapping the view frustum into a unit cube
- a **perspective divide** is applied after clipping, where the x , y , and z coordinates are divided by the homogeneous coordinate w , which encodes depth information generated by the projection matrix
 - this causes objects further away to appear smaller, creating the illusion of depth

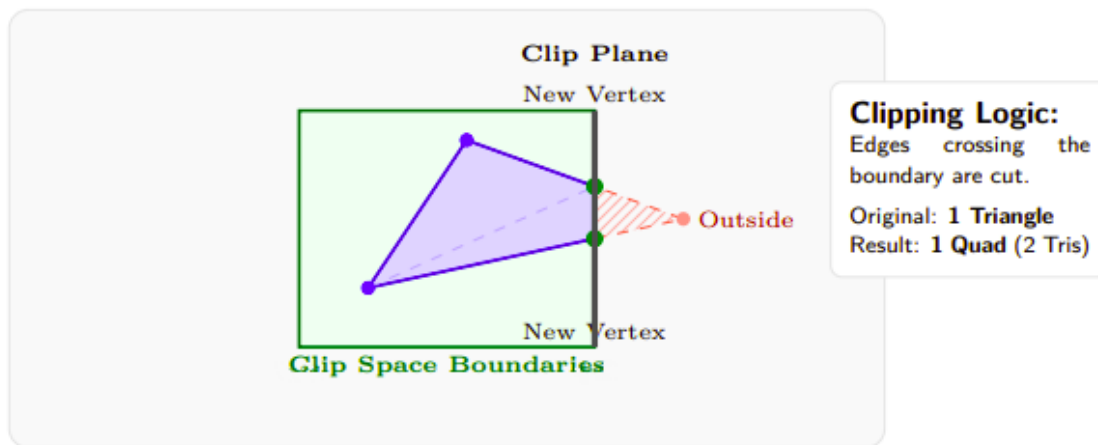


1.1.3 Clipping

Clipping is the process of removing geometry that lies outside the camera's view (view frustum) to reduce the amount of processing needed.

The GPU checks whether vertices/primitives lie within the viewing volume, keeping only those that are within the view:

- if a triangle is partly inside and partly outside the view frustum, it cannot be discarded (as part is visible) but it cannot be rendered in full (as part is outside the view)
- the solution is to cut the triangle along the clipping boundary and create new vertices where edges intersect the frustum, which forms new triangles which fit entirely within the view volume
 - a common algorithm that does this is Sutherland-Hodgman



Clipping is performed after the projection matrix is applied and before the perspective divide, as perspective projection transforms the view frustum into a unit cube in clip space. This makes it easier to check if points lie within the cube; simply check that:

- $-w \leq x \leq w$
- $-w \leq y \leq w$
- $-w \leq z \leq w$

1.1.4 Backface culling

Backface culling is the process of discarding polygons that face away from the camera, since they are not visible in a closed 3D object, so rendering them is computationally wasteful.

To determine if a surface is facing the camera, a dot product can be used:

$$\vec{V} \cdot \vec{N}$$

where: - $\vec{V}$ is the view direction - $\vec{N}$ is the surface normal vector

If the dot product is positive, the face is pointing away, so the surface is discarded. If the dot product is negative, the face is pointing towards the camera, and is kept.

A slightly more efficient way of determining whether triangles are back-facing is by checking **winding order** after projection:

- winding order determines whether a triangle is front- or back-facing by checking whether its vertices appear clockwise or counter-clockwise in screen space
- one winding direction is defined as front-facing and the opposite is treated as back-facing, allowing fast culling without computing normals

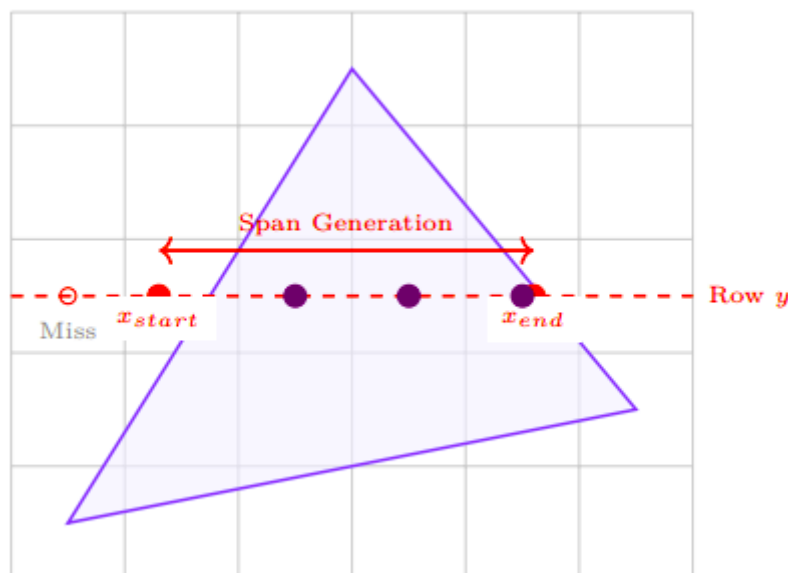
1.1.5 Rasterisation

Rasterisation is the process of converting continuous 2D geometric shapes (triangles) into discrete screen fragments (potential pixels).

After clipping and culling, geometry is still made of mathematical lines and triangles. However, the screen is a grid of pixels.

Rasterisation uses a **scanline algorithm** to determine which pixels are covered by each triangle:

- triangle vertices are sorted by y -coordinate to find the top, middle and bottom
- calculate the equation for the left and right edges
 - these are typically linear, in the form $Ax + By + C = 0$
- for each horizontal row (scanline) in the pixel grid,
 - compute:
 - * x_{start} : the x-coordinate where the scanline intersects the left edge
 - * x_{end} : the x-coordinate where the scanline intersects the right edge
 - the pixels from x_{start} to x_{end} form a **span**
 - * each pixel centre inside the span becomes a fragment
 - * a fragment is a potential pixel, containing screen-space position (x, y) , depth (z -value) and attributes such as colour and texture
 - attributes are linearly interpolated between x_{start} and x_{end} to compute per-fragment values
 - generally, a pixel is only filled if its centre point lies inside the triangle



Because pixels are discrete, triangle edges look jagged. This is called **aliasing**. It occurs because continuous shapes are approximated using a discrete pixel grid.

1.1.6 Hidden surface removal (HSR) and shading

The final stage of the rendering pipeline determines:

- Which fragments are visible (Hidden Surface Removal)
- What colour they should be (Shading)

Shading

Shading is performed by the **fragment shader**.

The fragment shader calculates the final colour of each fragment using:

- surface normals
- lighting (light sources, direction, intensity)
- textures (UV mapping)

For each fragment, it outputs a final pixel colour.

Hidden surface removal

Not all fragments should be drawn - some are hidden behind others.

Painter's algorithm The historical approach was the Painter's algorithm:

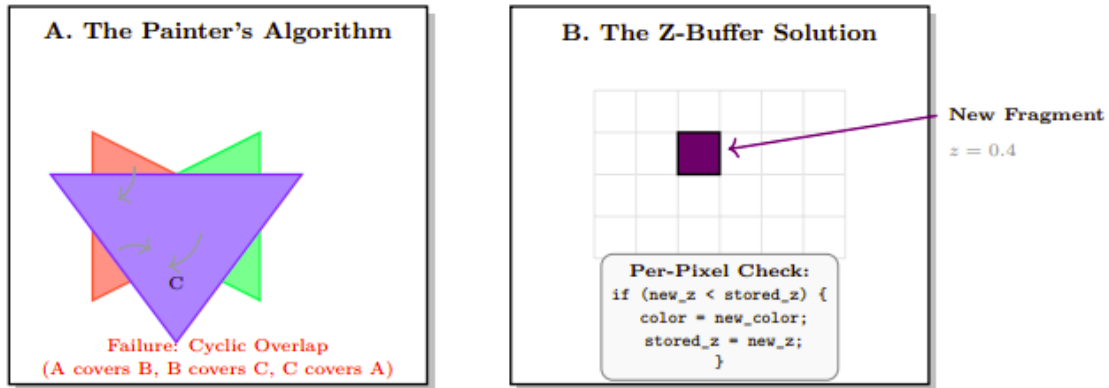
- sort polygons by depth (approximate back-to-front ordering)
- draw far objects first, then paint nearer ones on top

However, this method had problems:

- it was slow, since sorting is $O(n \log n)$
- it fails completely for cyclic overlap, where three triangles overlap each other in a cycle (A in front of B, B in front of C, C in front of A)

Z-buffer The modern solution is to use a **Z-buffer (depth buffer)**:

- the GPU maintains the depth buffer, which is a 2D array of floats matching the screen resolution that stores the depth of the closest fragment seen so far
- at the start of every frame, the depth buffer is cleared to the maximum distance (e.g., $z = \infty$)
- then, for each frame, for each fragment at position (x, y) :
 - if $Z_{new} < Z_{stored}$:
 - * update depth buffer with Z_{new}
 - * write fragment to frame buffer (stores final pixel colour output of the image)
 - else, discard the fragment (as it is hidden)



This method is:

- fast - constant ($O(1)$) check time per pixel
- order independent - triangles can be processed in any order
- precise - visibility is decided per pixel (can handle cyclic overlaps)
- highly parallelisable - depth testing is performed independently per fragment, making it ideal for GPU architecture

However, a possible problem can occur if the depth buffer has limited precision:

- **Z-fighting** is when the GPU cannot reliably distinguish which fragment is closer, causing flickering

1.2 GPU memory architecture

Beyond the processing pipeline, the GPU acts as a massive, high-speed memory manager.

It manages distinct regions of VRAM (Video Random Access Memory) called **buffers**. Two of the most important things that it must manage are:

- **time** (to ensure the image is visually stable, with no flickering/tearing)
- **space** (to ensure the image is geometrically correct i.e., proper overlap of objects)

To manage space, the GPU uses Z-buffering, which was covered [above](#).

1.2.1 Managing time: double buffering

- the monitor refreshes line-by-line, from top to bottom
- if the GPU writes new pixels while the monitor is refreshing, the top of the screen may show the previous frame, while the bottom shows the new frame
- this is called **screen tearing**

The solution to this is to use **double buffering**, which decouples rendering from display, ensuring only complete frames are shown:

- separate GPU writing and display reading using two buffers:
 - **back buffer** (private): where the GPU renders the next frame (never visible)
 - **front buffer** (public): contains the completed frame read by the display
- when the GPU finishes drawing the back buffer

- the buffers are swapped at the next **VSync**, or Vertical Sync (end of monitor refresh cycle)
 - * back buffer becomes the new front buffer
 - * GPU begins clearing and drawing to the old front buffer (new back buffer)

This prevents tearing, but may introduce input latency due to waiting for VSync.

1.2.2 Memory cost

For each pixel, the buffers required are:

- front buffer (RGBA): 4 bytes
- back buffer (RGBA): 4 bytes
- depth buffer: 4 bytes

Therefore, the minimum VRAM required to render a frame is approximately given by:

$$\text{VRAM} \approx \text{Pixels} \times 12 \text{ bytes}$$

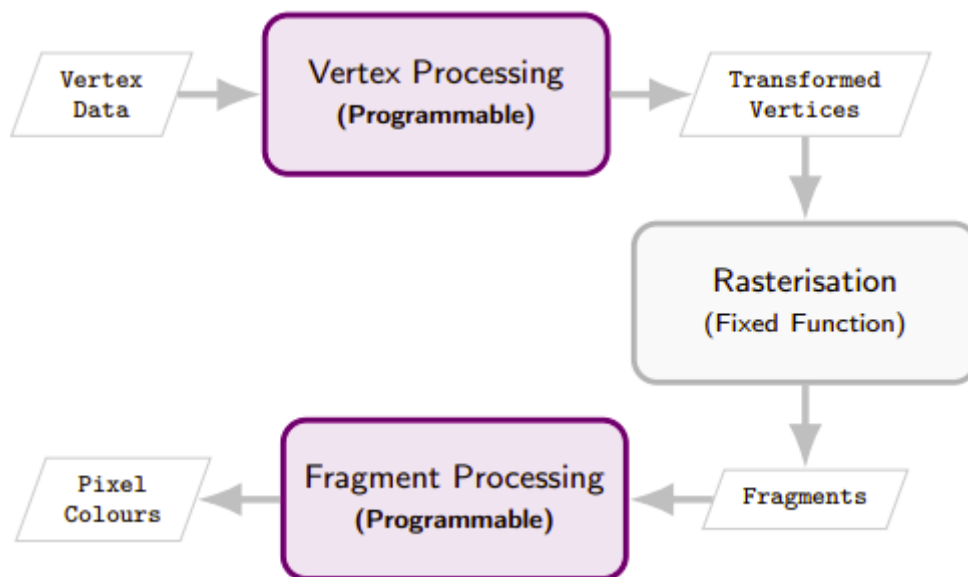
For 1080p (~2 million pixels), this is ≈ 24 MB.

Note that this only the bare minimum cost to just open a window with the given number of pixels, before any textures, models etc. are loaded, which would significantly increase the required memory.

1.3 The modern programmable pipeline

Modern APIs like WebGL abstract the rendering pipeline into three main conceptual stages, centred on the programmable parts:

1. Vertex processing (programmable)
2. Rasterisation (fixed/ non-programmable)
3. Fragment processing (programmable)



1.3.1 Vertex processing

Vertex processing is the first programmable stage. Its primary purpose is to process the individual points (vertices) that make up the 3D models.

The GPU does this using the **vertex shader**:

- written by the programmer
- runs once for every vertex in the scene
- responsible for:
 - **transformation**: transforming vertex positions from model space through world and camera space into clip space (later converted to screen coordinates)
 - * involves a series of matrix multiplications that account for the model's position, the camera's viewpoint, and the perspective effect
 - **data passthrough**: passing per-vertex data (e.g. texture coordinates, normals) to the next stage

1.3.2 Rasterisation

Rasterisation is a fixed, non-programmable part of the hardware. It takes transformed vertices from the previous stage and determines which pixels on the screen are covered by the geometric shapes (primitives) they form:

- vertices are assembled into primitives (usually triangles)
- rasteriser fills in these triangles, iterating over the 2D grid of pixels and generating a fragment for each pixel that lies inside the triangle's boundary
- per-vertex attributes (e.g. colour) are smoothly interpolated across the surface of the triangle for each fragment

1.3.3 Fragment processing

Fragment processing is the second programmable stage.

The GPU performs this using the **fragment shader**:

- written by the programmer
- runs once for every fragment generated by the rasteriser
- computes the final colour of the fragment
- uses the interpolated data from the rasteriser to perform tasks like texturing and lighting

After this stage, per-fragment operations (e.g. depth testing) determine whether the fragment is written to the framebuffer (stores the colour and related data of pixels for a rendered frame before it is displayed).

1.4 The polygon model

The **polygon model** represents 3D objects using meshes made of polygons, almost always triangles. Triangles are the native primitive of the GPU.

There are several reasons for using triangles:

- **guaranteed flatness**:
 - a triangle is the simplest polygon, defined by exactly three points

- any 3 points in 3D space are always coplanar (lie in the same plane)
- therefore, a triangle is always perfectly flat
- **well-defined normals:**
 - a triangle has a single constant normal vector $\vec{N}$
 - this normal is perpendicular to the surface and constant across the triangle (since the triangle is flat)
- **precise lighting:**
 - lighting calculations depend on the surface normal
 - since triangles have a constant normal, lighting is simple and predictable

Another polygon, like a quad (4 vertices) is not guaranteed to be flat. If it is non-coplanar (bent), different parts of the surface face different directions, so it does not have a single consistent normal, leading to ambiguous lighting. In practice, quads are split into two triangles.

Modern GPUs are designed and optimised to process triangles; the rasterisation process assumes triangle input.

1.4.1 Triangle reduction

Rasterisation (converting shapes to pixels) and shading (calculating colour/light per pixel) are the most expensive stages of rendering. Therefore, the number of triangles reaching these stages should be minimised as early as possible in the pipeline. This is achieved using stages such as **clipping** and **backface culling**.

1.5 Transformations

A **transformation** is a mathematical operation that changes the position, rotation, or scale of geometric data.

Transformations are needed as 3D models are just stored as static numbers, so to create a dynamic, interactive world, this data needs to be moved through different coordinate spaces (e.g. model $\rightarrow$ world $\rightarrow$ view $\rightarrow$ clip space).

A naive approach may be to modify the vertex data directly on the CPU by iterating through each vertex and applying an operation. However, this is massively inefficient for many reasons:

- **High CPU Cost:** processing millions of vertices per frame is expensive, and the CPU must also handle tasks like game logic and physics
- **Slow Data Transfer:** sending updated vertex data to the GPU every frame is limited by bus bandwidth
- **Loss of Original Data:** modifying the data directly overwrites the original model, making reuse difficult

A much more efficient method is to perform transformations on the GPU in the vertex shader:

- the original vertex data is sent to the GPU once
- each frame, **transformation matrices** are sent to the GPU as uniform variables
 - these matrices encode transformations and are used to transform vertex positions
- the vertex shader applies these matrices to every vertex in parallel

This approach offloads massively parallel computation to the GPU, which is designed for it (having thousands of cores) and minimises the data transfer between the CPU and GPU.

1.6 Fundamental 2D transformations

The three fundamental 2D transformations that form the basis of most 2D graphics operations are:

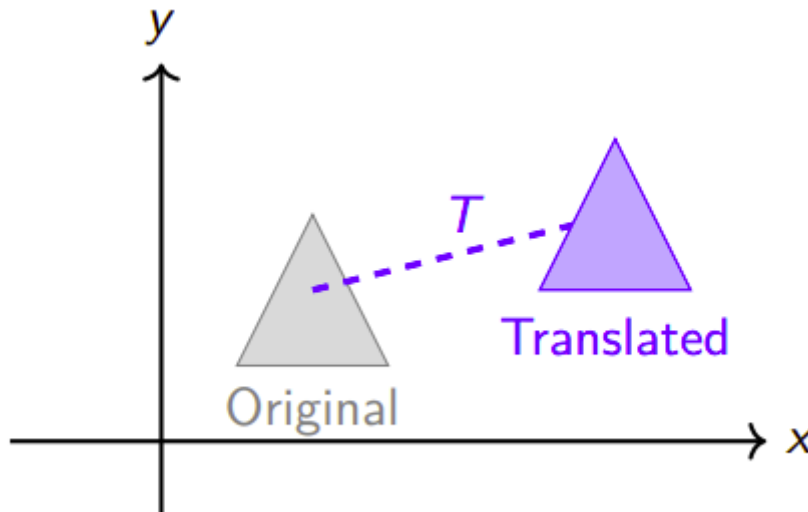
- translation
- rotation
- scaling

1.6.1 Translation (moving)

Translation is the process of moving an object from one position to another. It is defined by a translation vector $T = (t_x, t_y)$, which specifies the displacement in the x and y directions.

For any vertex $P = (x, y)$, the translated vertex P' is calculated by simple vector addition:

$$P' = P + T = (x + t_x, y + t_y)$$



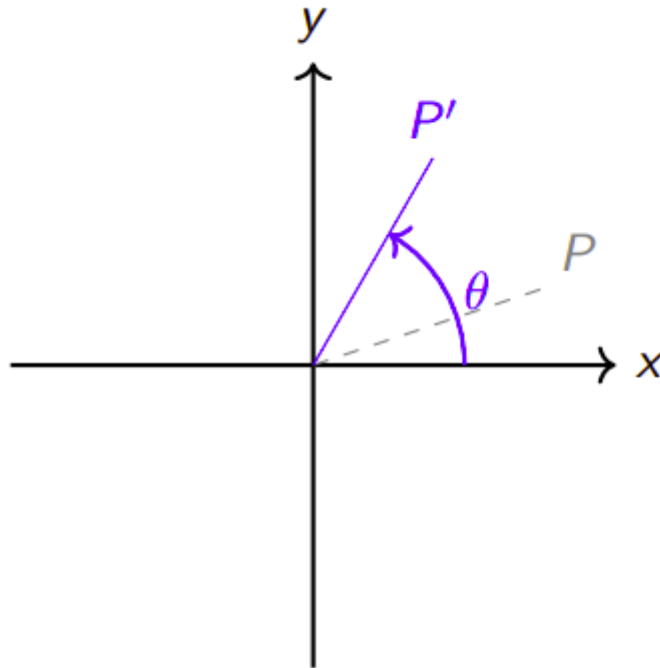
1.6.2 Rotation (turning)

Rotation is the process of turning an object around a specific point, known as the pivot point. For simplicity, we first consider rotation around the origin $(0, 0)$.

To rotate a vertex $P = (x, y)$ by an angle θ anti-clockwise, the following trigonometric formulae are used to find the new vertex $P' = (x', y')$:

$$x' = x \cos \theta - y \sin \theta$$

$$y' = x \sin \theta + y \cos \theta$$

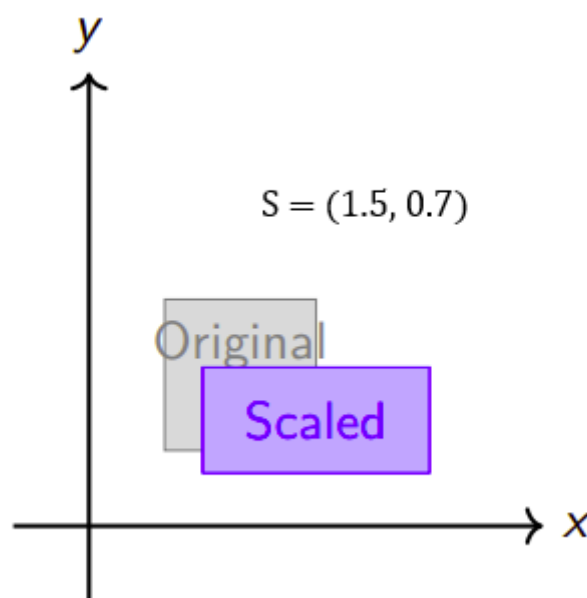


1.6.3 Scaling (resizing)

Scaling is the process of changing the size of an object. It is defined by a scaling vector $S = (s_x, s_y)$. For a vertex $P = (x, y)$, the scaled vertex $P' = (x', y')$ is found by component-wise multiplication:

$$x' = x \times s_x$$

$$y' = y \times s_y$$



If $s_x = s_y \sim s$, the scaling is uniform, preserving the object's aspect ratio. If $s_x \neq s_y \sim s$, the scaling is non-uniform, which will stretch or squash the object. Scaling is also performed relative to the origin; vertices move farther away from or closer to the origin based on the scaling factors.

1.6.4 Transformations as matrices

For each of the three transformations above, the equations use different operations (addition for translation, trigonometric functions for rotation, multiplication for scaling).

Matrices provide a single, consistent mathematical operation to represent all transformations: - each 2D point (x, y) can be represented as a column vector

$$\begin{pmatrix} x \\ y \end{pmatrix}$$

- the transformation is represented as a matrix
- the transformation is applied by performing matrix-vector multiplication

Scaling matrix

The scaling operation $x' = x \times s_x, y' = y \times s_y$ can be written in matrix form as:

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} s_x & 0 \\ 0 & s_y \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix}$$

Rotation matrix

The rotation operation $x' = x \cos \theta - y \sin \theta, y' = x \sin \theta + y \cos \theta$ can be written as:

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix}$$

The problem with translation

Using the matrices above, scaling and rotation are now a single, consistent operation: multiplication by a 2×2 matrix.

However, translation is an addition: $P' = P + T$. There is no 2×2 matrix M such that:

$$M \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} x + x_t \\ y + y_t \end{pmatrix}$$

This is because matrix multiplication is a linear transformation (origin stays fixed), but translation is an affine one (origin moves).

Homogeneous coordinates To solve this problem, **homogeneous coordinates** can be used:

- the idea is to extend 2D coordinates into homogeneous coordinates using an extra dimension
- a 2D point $P = (x, y)$ can be represented as a 3D vector by adding a third 'w' coordinate and setting it to 1:

$$\begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

- by moving to 3D vectors and 3×3 matrices, the third column of the matrix can be used to encode translation
- this allows affine transformations in 2D (like translation) to be represented as linear transformations in homogeneous coordinates
- w acts as a scale factor, where the ‘real world’ is the slice where $w = 1$
- changing w scales the coordinate values, but represents the same location in space (once normalised):
 - any point (x, y, w) is equivalent to $(x/w, y/w, 1)$ - they lie on the same projective ray

Each transformation can now be represented as a 3×3 matrix which is multiplied with the homogeneous coordinate vector.

Translation matrix (homogeneous)

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

Rotation matrix (homogeneous)

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

Scaling matrix (homogeneous)

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

1.6.5 Combining transformations

The key benefit of this unification is that it allows multiple transformations to be combined into a single matrix via matrix multiplication.

It is very important to note that the order of matrix multiplication matters, as transformations are not commutative. In general, for matrices A and B , $AB \neq BA$.

Transformations are applied from right to left (when using column vectors):

- for example, to first scale and object, then rotate it, and finally translate it, the matrices are applied in that order:

$$P' = M_{translate} \times M_{rotate} \times M_{scale} \times P$$

This is the conventional order of operations for placing on object (scale $\rightarrow$ rotate $\rightarrow$ translate). This ensures the object scales in its own local space, rotates around its own centre, and then moves to its final world position.

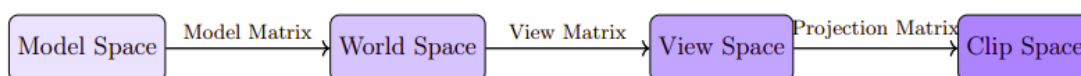
The combined matrix can be computed once and applied to all vertices efficiently.

1.7 3D transformations

Moving from 2D to 3D is a natural extension of the above concepts. The principles remain the same, just with an extra dimension added:

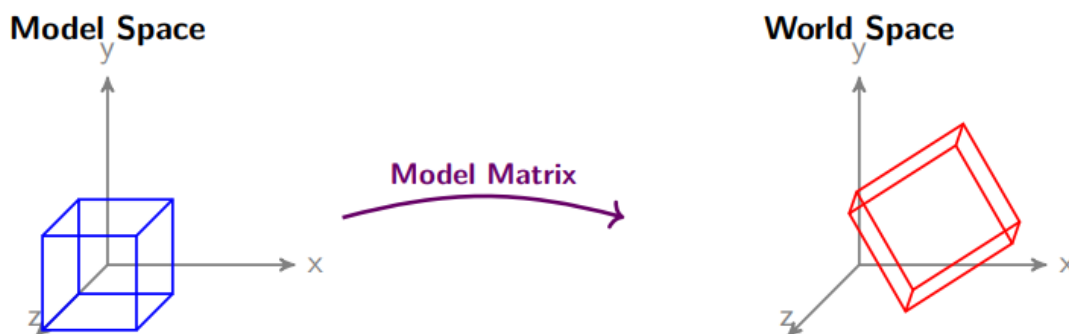
- vertices now have three components: (x, y, z)
- homogeneous coordinates are used by adding a w component, making the vectors 4D: $(x, y, z, 1)$
- the transformation matrices are now 4×4

The process of taking a 3D model and rendering it onto a 2D screen is a journey through several distinct coordinate systems. This journey is orchestrated by a sequence of three critical matrices:



1.7.1 The model matrix

The **model matrix** is responsible for taking a model's vertices in its local coordinate system (model space) and positioning them within the larger world. It applies a specific translation, rotation, and scale to the object to set its size, orientation, and position in the overall scene, transforming the vertices from model space into world space. Every object in a scene will have its own unique model matrix.



The model matrix is a combination of the 3D transformation matrices in homogeneous coordinates, which are direct extensions of their 2D counterparts. Rotation in 3D is more complex, as it can occur around the X, Y, or Z axes, so there are three matrices for rotation.

$$M_{translate} = \begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$M_{scale} = \begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$R_x(\theta) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$R_y(\theta) = \begin{pmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$R_z(\theta) = \begin{pmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

The model matrix is then typically of the form $M = T \times R_z \times R_y \times R_x \times S$. Remember that this mean scaling is performed first, and translation last.

1.7.2 The view matrix

The view matrix transforms coordinates from world space into view space (also called camera space). It represents the position and orientation of the camera in the scene.

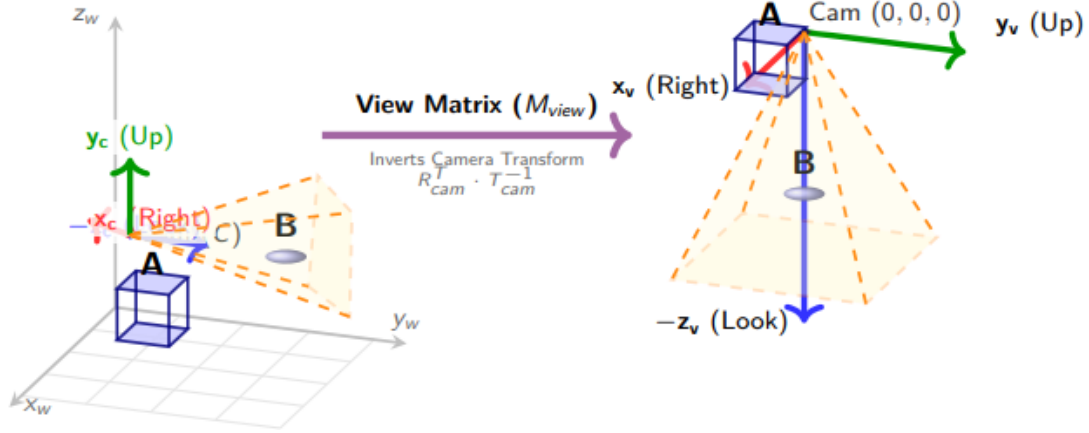
Instead of moving the camera, the view matrix moves the entire world to position it in front of a fixed camera.

In view space:

- the camera is at the origin (0, 0, 0)
- it looks down the negative Z-axis ($-Z$)

2. View Space

1. World Space



Suppose that the camera is positioned at position $C = (C_x, C_y, C_z)$ within the world space. Then, to put the camera at the origin, the world is translated by $-C$.

Next, the world is rotated so that it matches the camera's orientation. A new coordinate system is defined based on the camera's perspective. This is done using three orthonormal basis vectors: forward, right, and up.

Given:

- Camera position C
- Target point T (what the camera is looking at)
- World up vector $\vec{U}_{p_{world}}$ (defines the global 'up' direction, typically the y-axis)

The camera basis vectors are constructed as follows:

$$f = \text{normalise}(T - C)$$

- The forward vector f defines the viewing direction of the camera (corresponding to $-Z$ in view space)

$$r = \text{normalise}(f \times \vec{U}_{p_{world}})$$

- The right vector r is perpendicular to both the forward direction and world up, defining the camera's x-axis

$$u = \text{normalise}(r \times f)$$

- The true up vector u is perpendicular to both the right and forward vectors, defining the camera's y-axis

The view matrix combines:

- the inverse rotation (camera orientation)

- the inverse translation (camera position)

It is given by:

$$V = \begin{pmatrix} r_x & r_y & r_z & -\mathbf{r} \cdot \mathbf{C} \\ u_x & u_y & u_z & -\mathbf{u} \cdot \mathbf{C} \\ -f_x & -f_y & -f_z & \mathbf{f} \cdot \mathbf{C} \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

1.7.3 The projection matrix

The projection matrix transforms view space into clip space, applying perspective or orthographic projection to map 3D coordinates into a 2D screen representation.

It is responsible for:

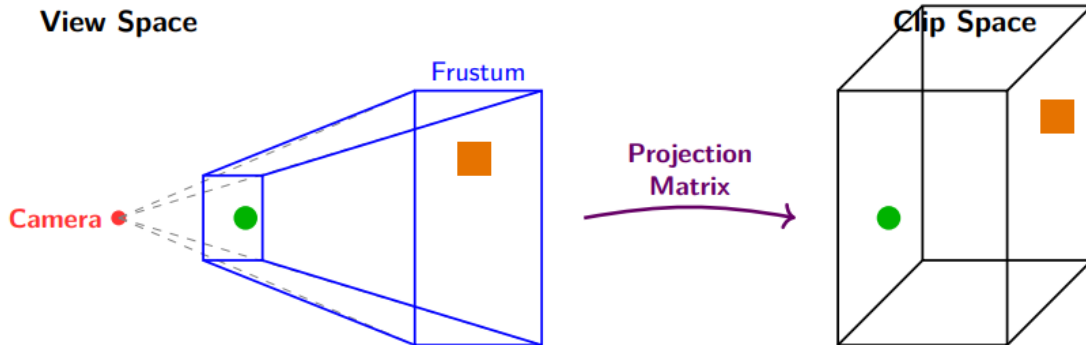
- defining the viewing volume - determining the region of the scene that is visible
- transforming vertices into clip space, preparing them for the perspective divide

After this, a perspective divide is performed:

$$(x, y, z, w) \rightarrow \left(\frac{x}{w}, \frac{y}{w}, \frac{z}{w}\right)$$

This produces Normalized Device Coordinates (NDC), where:

- $x, y, z \in [-1, 1]$ for visible geometry
- anything outside this range is clipped



Perspective projection

- simulates real-world vision
- the viewing volume is a frustum
- objects farther away appear smaller
- parallel lines converge
- used in most 3D games and realistic rendering

During projection:

- the transformation produces a w component related to depth ($w \propto -z$)
- dividing by w creates the perspective effect

Orthogonal projection

- viewing volume is a rectangular box
- no perspective distortion (size does not change with distance)
- parallel lines remain parallel
- used for diagrams, architectural plans, or 2D-style games
- $w = 1$ so no perspective divide scaling occurs

1.7.4 MVP matrix

The three main matrices (model, view and projection) can be multiplied together to form a Model-View-Projection (MVP) Matrix:

$$M_{MVP} = M_{Projection} \times M_{View} \times M_{Model}$$

Transformations are applied from right to left, meaning a vertex is first transformed into world space, then view space, and finally clip space:

$$v_{clip} = M_{MVP} \cdot v_{model} = M_{Projection} \cdot M_{View} \cdot M_{Model} \cdot v_{model}$$

Three.js example

```
// Scene holds all objects , lights , and cameras
const scene = new THREE.Scene() ;

// Camera defines the VIEW and PROJECTION
// PerspectiveCamera (fov , aspect , near , far)
// defines the PERSPECTIVE matrix
const camera = new THREE.PerspectiveCamera(75 , w/h , 0.1 , 1000);

// An object is a " Mesh " (Geometry + Material)
const geometry = new THREE.BoxGeometry(1 , 1, 1);
const material = new THREE.MeshNormalMaterial();
const cube = new THREE.Mesh(geometry, material);
scene.add(cube);

// These properties are used to build the MODEL matrix
cube.scale.set(1.5 , 1, 1);    // S
cube.rotation.y = Math.PI / 4; // R
cube.position.x = 2;          // T

// These properties are used to build the VIEW matrix
camera.position.set(1 , 2, 5);
camera.lookAt(cube.position); // Point camera at the cube

// The animate function is called every frame
function animate() {
    renderer.render(scene, camera);
```

```

    requestAnimationFrame(animate);
}

```

When `renderer.render(scene, camera)` is called, three.js performs a number of steps:

1. It checks `cube.position`, `cube.rotation`, `cube.scale` and computes the final Model matrix for the cube.
2. It checks `camera.position` and orientation (`camera.lookAt`) to compute the View matrix.
3. It uses the camera's settings (FOV, etc.) to get the Projection matrix.
4. It computes the final MVP matrix ($P \cdot V \cdot M$).
5. It sends the MVP matrix and vertex data to the GPU and tells it to draw.

1.7.5 Procedural terrain & object placement

Procedural terrain is a method of creating terrain using mathematical functions instead of manual modelling:

- it starts with a flat plane (grid of vertices)
- the height of each vertex is modified using a function
 - e.g., $y = \sin(x) \cdot \cos(z)$
 - this is called vertex displacement
- this produces hills and valleys automatically

Raycasting is a technique where a ray (line) is projected into a scene to detect intersections with objects. It is commonly used to determine where objects should be placed.

The equation of a ray is:

$$\vec{R}(t) = \vec{O} + t\vec{D}$$

where:

- O is the starting point (origin)
- D is the direction
- t is the distance along the ray

To detect where to place an object:

- a ray is cast downwards from above the terrain
- the point where it intersects the terrain is calculated
- the object is moved to that point
 - this ensures the object sits exactly on the surface

Three.js example

```

// 1. Create Hills (Displace Y)
// get list of vertices in plane
const pos = plane.attributes.position;

// for each vertex, set its Y value using a sine function
for (let i = 0; i < pos.count; i++) {
  let y = Math.sin(pos.getX(i)*0.3) * 1.5;
  pos.setY(i, y);
}

```

```

}

// 2. Raycast
raycaster.set(
    new THREE.Vector3(x, 10, z), // origin of ray (above point)
    new THREE.Vector3(0, -1, 0)  // direction of ray (down)
);

// find intersections with terrain
const hits = raycaster.intersectObject(terrain);

// place object at the intersection point
if (hits.length > 0) {
    house.position.y = hits[0].point.y;
}

```

1.8 Animation

1.8.1 Transformation consistency

To create motion, transformation values are updated inside the render loop. The method used to calculate these updates is important:

- **frame-based animation**
 - `position += speed`
 - `speed` is measured in units per frame
 - the animation becomes dependent on hardware performance
 - a higher framerate causes the object to move faster (since the render loop is applied each frame)
- **time-based animation**
 - `position += speed * deltaTime`
 - `speed` is measured in units per second
 - `deltaTime` is the time since the previous frame
 - the animation speed is consistent across all frame rates

1.9 Lighting and shading

1.9.1 Appearance and materiality

A **Material** controls how a 3D surface interacts with light:

- it defines the object's visual appearance, not its shape
- the Mesh (geometry) controls shape, the Material controls look

Within the material:

- **Albedo** is the base colour of the material
- it represents the object's intrinsic colour (no lighting applied)
- it is used as the input to lighting calculations

In Three.js:

- colour is a property of the Material, not the geometry
- the `THREE.Color` class handles colour formats (Hex, RGB, HSL)

The code snippet below demonstrates creating a colour and assigning it as a property of a mesh:

```
const myCol = new THREE.Color(0x660066); // albedo
const mat = new THREE.MeshPhongMaterial({
  color : myCol,
  specular : 0x111111 // controls shininess
});
```

To work with lighting, the material's albedo acts as a multiplier for incoming light:

$$\text{Final Pixel} = \text{Albedo} \times (\text{Ambient} + \text{Diffuse} + \text{Specular})$$

where:

- Ambient is the general lighting in the scene
- Diffuse is the light hitting the surface
 - depends on the angle between the light direction and surface normal
 - brightness (diffuse intensity) is highest when they are aligned
- Specular is any shiny highlights
 - brightest when the view direction aligns with the reflection direction (**Phong**)
 - or when the normal aligns with the halfway vector (**Blinn-Phong**)

Albedo modulates (tints) incoming light; without light, the object appears black.

1.9.2 HSV/HSL colour space

HSV/HSL colour spaces are designed to match how humans perceive colour by separating tint, intensity, and brightness.

- Hue (H): The colour's tint on a 0–360° colour wheel
 - e.g. red, green, blue
- Saturation (S): The vibrancy/purity of the colour
 - High S → vivid colour, Low S → washed out/grey
- Value/Lightness (V/L): The brightness of the colour
 - Low → dark, High → bright

There are some advantages to using this colour space:

- procedural jittering
 - introduce small variation between objects
 - keep S and V/L the same, and vary H
 - objects look consistent, but not identical
- intuitive shading
 - create natural shadows without changing the hue
 - keep H the same, reduce V/L

In Three.js, colours can be set using HSL via `THREE.Color`:

```
const col = new THREE.Color();
// Hue is normalised to between 0 and 1: 0.1 36°
col.setHSL(0.1, 0.8, 0.5); // (H, S, L)
```

1.9.3 Lighting

In computer graphics, **lighting** is the mathematical simulation of how light interacts with object surfaces (meshes). Algorithms compute the colour and brightness of each pixel based on light sources and material properties.

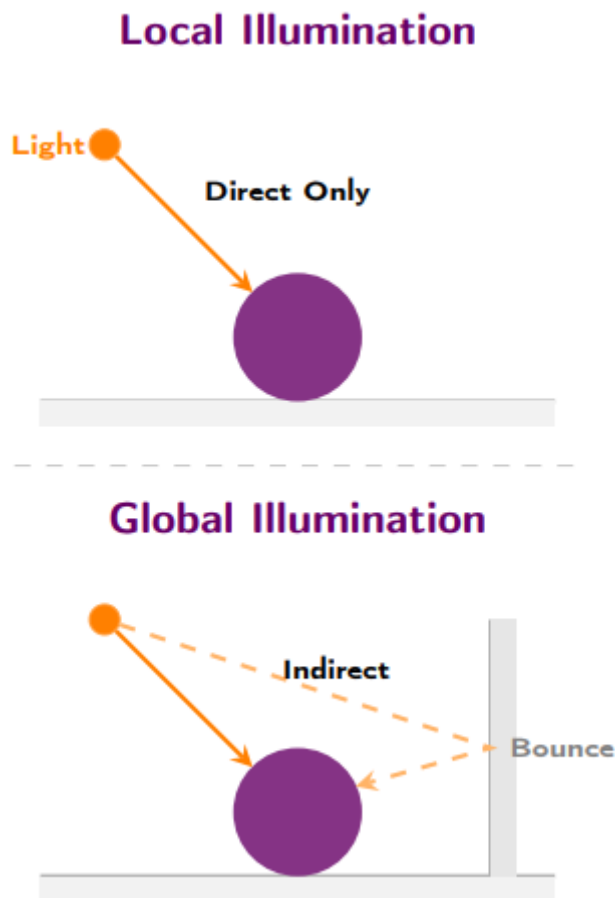
Lighting is needed because:

- without lighting, a 3D sphere is indistinguishable from a flat 2D circle
- it helps convey material properties, through diffuse shading and specular highlights
- shadows and gradients provide depth cues, creating the illusion of 3D

Global vs Local illumination

Illumination models describe how light is calculated in a scene:

- **Global Illumination** (GI) simulates indirect lighting, where light bounces between surfaces
 - Physically accurate but extremely computationally expensive
- **Local Illumination** calculates light for an object using only direct light sources
 - Ignores indirect light between surfaces
 - Each object is shaded independently
 - Standard rasterisation-based rendering (like OpenGL) relies on Local Illumination



Ambient lighting is often used to fake indirect global light. This involves applying a constant, low-intensity light to every point on every object equally, regardless of orientation. It helps prevent scenes from appearing completely dark in shadowed areas.

Lighting vs Shading models

A lighting model is a mathematical approximation of how light interacts with a surface at a single point. It computes colour using vectors for light direction, surface normal and view (camera) direction.

A shading model defines how the lighting model is applied across a surface. It determines where lighting is calculated and how frequently it is evaluated. This creates a performance vs quality trade-off.

Normal vectors for the lighting equation A normal vector is a vector perpendicular to a surface. It is used in lighting calculations to determine how light interacts with the surface.

There are two approaches to calculate the surface normal of an object:

- per face (flat)
 - a single normal is used for the entire polygon
 - calculated using the cross product of two edges
- per vertex (smooth)
 - each vertex has its own normal
 - calculated by averaging the normals of all faces sharing that vertex

In Three.js,

```
geometry.computeVertexNormals();
```

can be used to automatically calculate vertex normals for a geometry/mesh.

Blinn-Phong lighting model

The **Blinn-Phong lighting model** is an empirical lighting model, meaning it approximates the appearance of lighting rather than simulating full physical accuracy. It:

- is computationally efficient
- produces visually good results
- is easy to control
- is widely used in real-time rendering

$$\text{Final Colour} = \text{Ambient} + \text{Diffuse} + \text{Specular}$$



The Phong lighting model:

- uses reflection vector ($\vec{R}$) for specular highlights
- compares $\vec{R}$ with view direction ($\vec{V}$)
- produces realistic highlights
- is more computationally expensive

The Blinn-Phong lighting model:

- uses halfway vector ($\vec{H}$) between light ($\vec{L}$) and view ($\vec{V}$)
- compares $\vec{H}$ with surface normal ($\vec{N}$)
- is more efficient than Phong
- produces highlights that are smoother and more stable

Directional lighting

Directional lighting represents a light source that is infinitely far away, such as the sun:

- Light rays arrive parallel to each other
- The light direction vector ($\vec{L}$) is constant across the entire scene
- The light has direction only, not position

As a result:

- there is no need to recompute the light direction per fragment
 - this improves efficiency in the fragment shader
- light intensity does not decrease with distance (no attenuation)

Point lighting

Point lighting represents a light with a specific position in 3D space:

- Light is emitted radially in all directions
- The light direction vector ($\vec{L}$) varies per fragment: $\vec{L} = P_{light} - P_{fragment}$
 - this is the vector from the surface point to light source
- Light intensity decreases with distance (attenuation)
- More computationally expensive than directional lighting (requires per-fragment calculations)
- The vertex shader transforms vertex positions into world/view space
- The fragment shader uses interpolated positions to compute light direction, distance to light and attenuation
 - The formula for attenuation is often based on the inverse square law $L \propto \frac{1}{d^2}$ (approximated/tuned for real-time rendering)

Updating normals

Lighting depends on the Normal vector ($\vec{N}$), which must remain perpendicular to the surface for correct light calculations.

Curved objects use per-vertex normals, which are interpolated across triangles, producing smooth lighting that hides flat geometry.

Non-uniform scaling (e.g., stretching only the X-axis) distorts the surface, which causes normals to no longer be perpendicular to the surface. As a result, lighting calculations become incorrect.

To fix this, normals are multiplied by the inverse transpose of the model matrix (the normal matrix):

$$\vec{N}_{\text{transformed}} = (M^{-1})^T \cdot \vec{N}_{\text{original}}$$

Shading model

The shading model determines how often the lighting equation is evaluated across a surface:

- Lighting calculations are computationally expensive
- More frequent evaluation produces better visual quality, but has a higher cost

Flat shading

- Lighting is calculated once per face
- Each triangle has a single, constant colour
- Produces a faceted appearance

Per-vertex shading (Gouraud)

- Lighting is calculated at each vertex
- Resulting vertex colours are interpolated across the triangle during rasterisation
- Produces smoother results, but can miss sharp highlights (if they occur inside a triangle)

Per-fragment shading (Phong)

- Vertex shader outputs normals for each vertex
- Normals are interpolated across each triangle
- Fragment shader performs the lighting calculation per fragment
- Produces smoother shading and accurate specular highlights

Per-fragment shading is the modern standard. It is computationally expensive, but GPUs are highly optimised to perform these calculations.

High-density lighting

Point lights are computationally expensive because lighting must be evaluated per fragment, and each additional light increases the number of calculations in the fragment shader.

Therefore, in scenes with many point lights, performance optimisation techniques are used, such as:

- Distance-Based Culling (Active Limits)
 - calculate the (squared) distance from every light to the camera

- sort the lights and only enable the closest ones (e.g., a maximum of 10 active lamps at a time)
- distant lamps are disabled
- Light Baking (Lightmaps)
 - for static objects (e.g., houses, trees, terrain), pre-calculate the illumination and save it into a 2D texture
 - shader samples this texture (using a second set of UV coordinates) instead of running light equations
 - real-time lights can be turned off entirely for static geometry

1.10 Geometry & Texture mapping

A mesh is the 3D structure that defines the shape of an object.

A mesh is made up of:

- Vertices: points in 3D space
- Edges: lines connecting vertices
- Faces: surfaces formed by connecting vertices (usually triangles)

Triangles are the most common primitives (basic geometric shapes) used in meshes because:

- they are the simplest polygon (defined by three vertices)
- they are always planar (flat), making calculations stable
- any complex polygon can be decomposed into triangles
- GPUs are highly optimised for rendering triangles

1.10.1 Vertex attributes

A vertex is a container for various attributes that describe the surface at a specific point:

- position: (x, y, z) coordinates in 3D space
- normal vector: a vector perpendicular to the surface, used for lighting calculations
- texture coordinates (UVs): 2D coordinates that map the vertex to a point on a texture (image)
- colour: a per-vertex colour value

This vertex data is sent to the GPU and stored in a dedicated memory buffer called a Vertex Buffer Object (VBO).

1.10.2 Index list

Edges are usually not stored explicitly in real-time rendering.

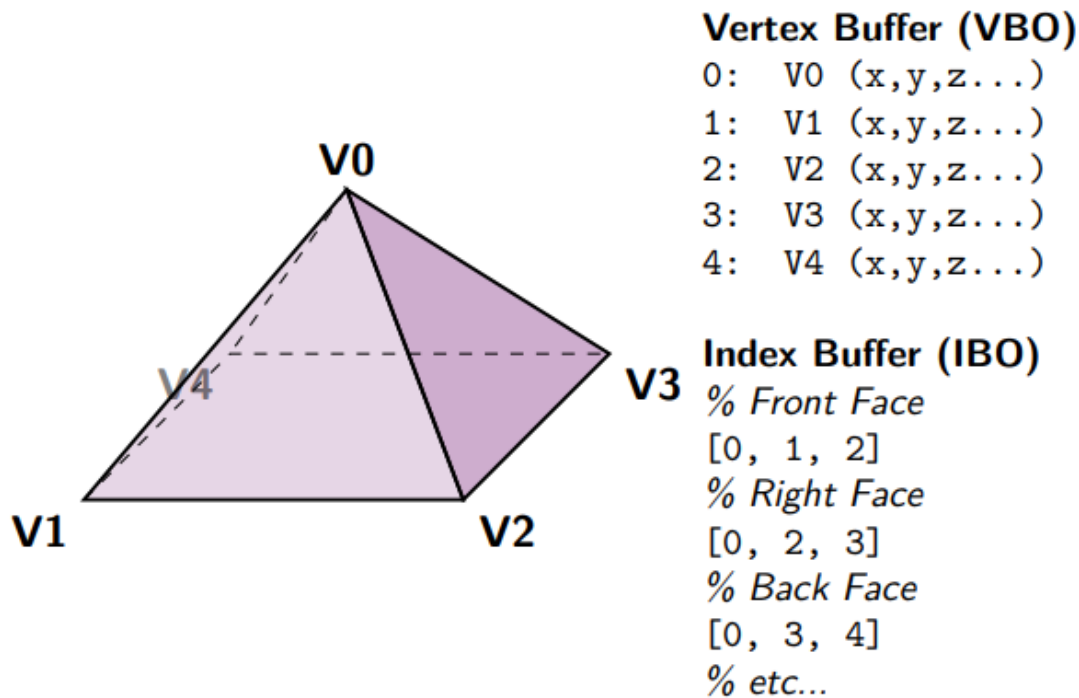
Instead:

- An index list stores integers that reference positions in the vertex list
- The GPU reads these indices in triplets to determine which vertices form each triangle (face)
- This index data is stored in a GPU memory buffer called an Index Buffer Object (IBO) or Element Buffer Object (EBO)

Using an index buffer allows multiple triangles to share the same vertices, which:

- reduces duplicated vertex data

- lowers memory (VRAM) usage
- improves rendering performance



In this example, vertex $V0$ is shared by four triangles. Instead of storing its data four times, it is stored once and referenced four times in the index buffer.

1.10.3 Texture mapping

Texture mapping is a function ϕ that maps a point from a 2D Texture Space (u, v) to a 3D Object Space (x, y, z) .

$$(u, v) \xrightarrow{\phi} (x, y, z) \xrightarrow{\text{Shader}} \text{Colour}$$

Texture mapping is needed because it adds visual detail without requiring additional geometry.

For example:

- a brick wall can be represented as a simple flat plane with a brick texture applied
- modelling every individual brick geometrically would require thousands more triangles

This improves realism while keeping rendering computationally efficient.

A **texel** (short for texture element) is the smallest unit of a texture image (essentially the texture equivalent of a pixel). It represents a colour stored in a texture space.

Forward and backward mapping

Forward mapping

- maps from the source (texture) to the destination (screen)
- iterates over texels in the texture image
- projects each texel onto screen space
- multiple texels may map to the same pixel (many-to-one)
- when scaling up (magnification), gaps (holes) can appear because some screen pixels may not be covered

Backward mapping

- maps from the destination (screen) back to the source (texture)
- iterates over screen pixels
- computes which point in texture space (UV coordinates) corresponds to each pixel
- samples the appropriate texel(s) to determine the pixel colour
- guarantees full screen coverage (no missing pixels)

Modern GPUs rely on backward mapping during the rasterization stage because it:

- ensures every screen pixel is shaded
- avoids gaps and overlaps
- works naturally with interpolation of vertex attributes (including UVs)

UV coordinates

u and v describe the 2D coordinate system of a texture image:

- u is the horizontal axis, ranging from 0.0 (left) to 1.0 (right)
- v is the vertical axis, ranging from 0.0 (bottom) to 1.0 (top)

Mapping

- each vertex stores a (u, v) pair as an attribute
- the rasteriser interpolates these values across the triangle's face
- the fragment shader uses the interpolated UV to sample the texture and determine the final pixel colour

Texture atlas

A texture atlas packs multiple smaller textures into one large texture image. This allows the GPU to render different surfaces in a single draw call by shifting the UV coordinate offsets for different vertices.

Texture modulation

A texture on its own is just a flat image representing surface properties (most commonly albedo, i.e. base colour). To make it appear as part of a 3D surface, it must interact with lighting.

This is done by treating the texture as the albedo term and modulating (multiplying) it by the light intensity calculated by the shading model.

$$C_{\text{final}} = C_{\text{texture}} \times (I_{\text{ambient}} + I_{\text{diffuse}} + I_{\text{specular}})$$

For each fragment, the steps the fragment shader must perform are:

1. Sample the texture at current UV
2. Calculate light intensity
3. Multiply texture colour by lighting intensity
4. Output the shaded pixel to framebuffer

Texture mapping in Three.js

```
// Texture loader handles logic for loading images
// and converting them into GPU textures
const textureLoader = new THREE.TextureLoader();

// Load the texture (returns a Texture object)
const colorTexture = textureLoader.load('path/to/my_texture.png');

// Create a material
// `map` assigns the texture as the material's albedo
const material = new THREE.MeshStandardMaterial({
  map: colorTexture, // base colour (albedo)
  roughness: 0.5     // controls light scatter
});

// Create mesh by combining geometry and material
const texturedMesh = new THREE.Mesh(geometry, material);

// Add mesh to scene
scene.add(texturedMesh);
```

Mip-mapping

When an object is far away, a single screen pixel can cover many texels in a texture. Sampling only the full-resolution texture in this case causes visual artifacts such as aliasing (shimmering or noise).

Mip-mapping is a solution to this:

- progressively lower resolution versions of the texture are precomputed and stored alongside the original texture
- each level (mip level) is a downscaled copy of the same image
- the GPU selects the most appropriate mip level based on distance

Aliasing

During rasterisation, the GPU attempts to represent continuous, infinite-resolution geometry using a discrete grid of pixels.

Each pixel is effectively a single sample, and its value is determined based on whether geometry covers it. This creates a sampling problem:

- a pixel is often treated as either fully inside or outside a triangle, based on the pixel centre

- this binary coverage test produces hard edges (jaggies)
- high-frequency detail (thin lines, sharp boundaries) is lost because the sampling rate (screen resolution) is too low to represent it accurately

Anti-aliasing reduces these artifacts by approximating coverage or filtering results. Common methods include:

- Supersampling (SSAA)
 - renders the scene at a higher resolution than the display
 - then downsamples (averages) to screen resolution
 - effectively increases sampling density
 - highest quality, but extremely slow
- Multisample AA (MSAA)
 - only increases sampling at triangle edges
 - fragment shader runs once per pixel (not per sub-sample)
 - stores multiple depth/coverage samples per pixel at edges
 - averages results to smooth boundaries
 - smooth edges, standard cost for textures
- Post-Process (FXAA)
 - blurs jagged edges after the frame is rendered
 - blurry, but very fast

Billboarding

A billboard is a flat 2D object (usually a quad made of two triangles) that is always rotated to the face the camera:

- the object's position stays fixed in world space
- its rotation is updated every frame
- it is oriented so its surface normal (or forward direction) aligns with the camera view direction

This can be used to simulate complex objects cheaply, reduce polygon count and maintain correct visual appearance from all angles.

Bump mapping

Bump mapping adds small-scale surface detail (such as bumps, dents, or wrinkles) without increasing geometric complexity or adding polygons.

It works by modifying how lighting reacts across the surface:

- a grayscale height map is used
 - lighter values represent higher areas
 - darker values represent lower areas
- the shader computes gradients in the height map across the surface
- these gradients are used to perturb (tilt) the surface normal used in lighting calculations

This creates the illusion of surface detail even though the underlying geometry remains unchanged.

Normal mapping

Normal mapping is a technique that adds the illusion of detailed surface geometry by storing modified surface normals in a texture. Like bump mapping, the actual mesh geometry does not change; only the lighting calculations change.

Instead of deriving normals from a height map at runtime, normal mapping stores the normals directly:

- surface normals are encoded per-pixel into a 2D texture
- each texel in the normal map stores a 3D normal vector, encoded as RGB values
 - red represents the X component
 - green represents the Y component
 - blue represents the Z component
- during rendering, the shader samples the normal map using interpolated UV coordinates
- the sampled RGB values are converted back into a normal vector
- this normal replaces or perturbs the mesh's original surface normal
- lighting calculations use the modified normal to create the illusion of detailed surface structure

A normal map can be generated from a grayscale height map by estimating surface gradients:

- a 3×3 neighborhood around each pixel is sampled
- horizontal and vertical slopes are computed
- the gradient is converted into a normal vector
- the vector is normalised and encoded into RGB values

Displacement mapping

Displacement mapping is a technique that physically moves vertices to change the geometry of the object:

- a grayscale displacement map (height map) is used to offset vertices along their normals:
 - black pixels keep the vertex at its original position
 - white pixels push the vertex fully outward along the normal
- the displacement map is sampled in the vertex shader
- instead of faking detail through lighting, the mesh itself is deformed

Environment mapping

Environment mapping is a technique used to simulate mirror-like reflections of the surrounding world on an object without using ray tracing:

- it gives the illusion that the object is reflecting its surroundings in real time
- instead of simulating full light transport, a cube map is used
 - a cube map is made of 6 textures, forming a box surrounding the scene
- to render:
 - reflection vector $\vec{R}$ is calculated
 - * based on the view direction and surface normal
 - * represents the direction a perfect reflection ray would travel
 - $\vec{R}$ is used to sample the cube map
 - * treated as a 3D direction vector from the center of the cube
 - * it is used to sample the appropriate face of the cube map based on direction

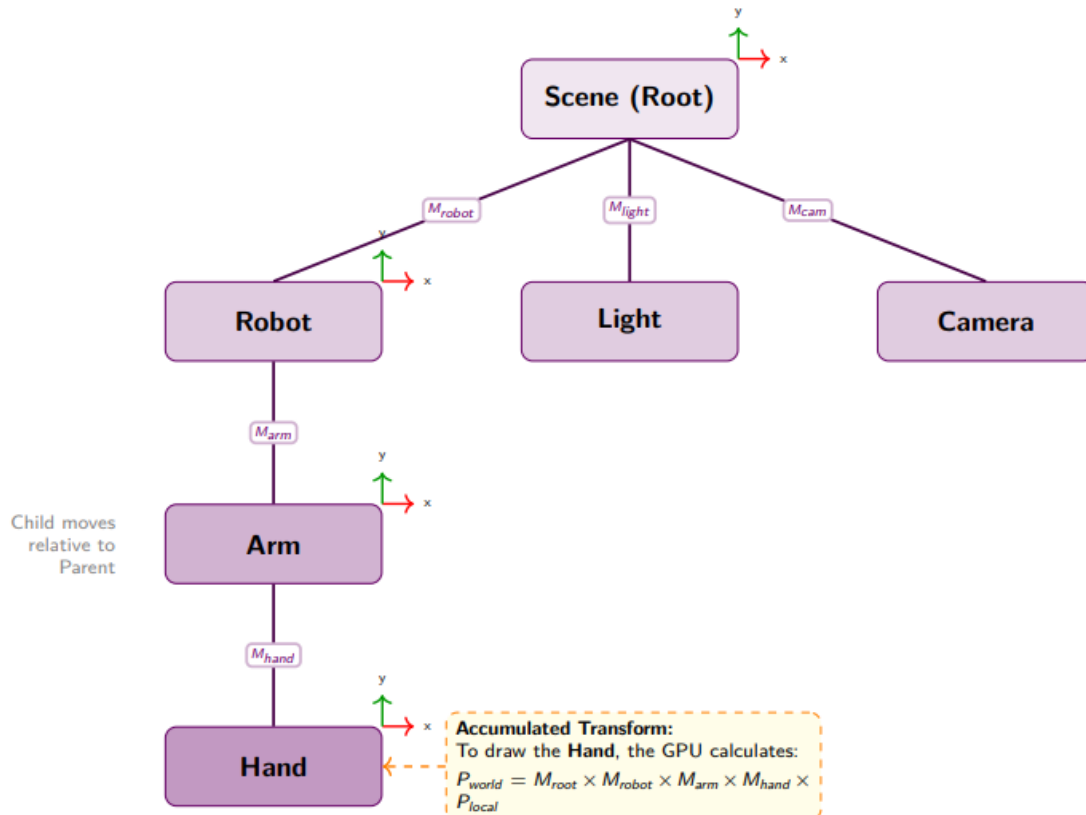
- the colour is fetched from the cube map
 - * the texel at that direction is read
- the colour is applied to the object surface
 - * used as the reflected colour of the fragment

1.11 Scene graphs

A (scene graph** is a hierarchical data structure used to organise the objects in a 3D scene by representing their relationships.

Instead of storing every object independently, objects are arranged in a directed acyclic graph (DAG), often structured similarly to a tree:

- the root node represents the whole scene
- child nodes inherit transformations from their parents
- each node can represent:
 - meshes/models
 - cameras
 - lights
 - groups
 - transformations
 - UI elements, etc.
- relationships are one way, from parent to child
- a node cannot be its own parent or ancestor, preventing infinite loops
- each edge represents a local transformation matrix

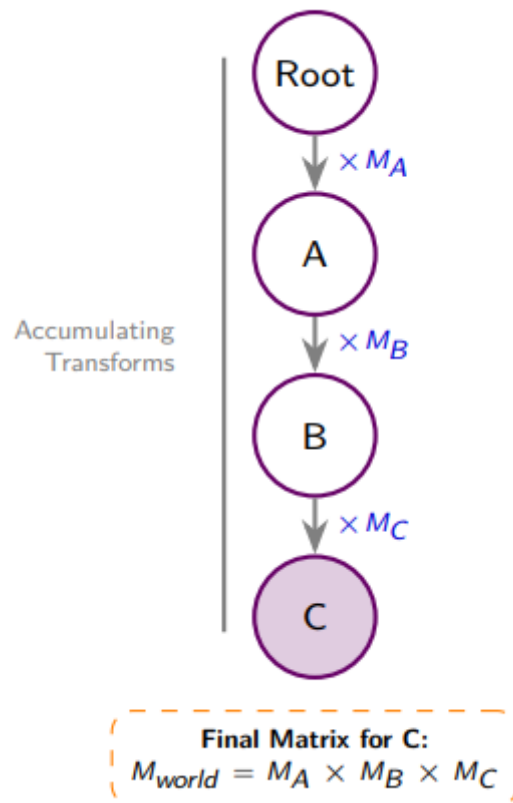


The example graph above shows a simple scene containing a robot with an arm and hand that can move independently. The final world transform of the hand is computed by multiplying all transformation matrices along the path from the root to the hand node.

1.11.1 Transformations in a scene graph

Each node stores its transformation (translation, rotation, scale) relative to its parent:

- this is its local transformation
- its final transformation in the scene is its world transformation
- an object's final world transformation is determined by its own local transformation and the transformations of all its ancestors
- a node's final world transformation is computed by multiplying transformation matrices up the hierarchy:
 - $M_{\text{world}} = M_{\text{parent_world}} \times M_{\text{local}}$
- this expands recursively all the way to the root node
- engines usually optimise this process by avoiding recomputation of child world matrices when parent transforms have not changed
 - if a node's local or world transformation changes, it and all of its descendants are marked as dirty
 - only nodes with a dirty flag have their matrices recomputed
- an additional optimisation is frustum culling
 - each parent node calculates a bounding box or bounding sphere that encapsulates all its children
 - if the parent's bounding volume lies outside the camera frustum, the entire branch can be discarded



Without a scene graph, every object would need to be transformed manually. A scene graph allows child objects to automatically inherit parent transformations, allowing complex objects and hierarchies to move together correctly.

1.11.2 Attatching a camera

Suppose an object is moving through a scene, and the camera needs to follow it:

- calculating the camera transformations manually so that it continuously tracks the object can become complex
- instead, the camera can be attached as a child node of the object in the scene graph
- the camera will then automatically inherit the object's transformations, causing it to move together with the object.
- the camera can still have its own local transformation relative to the parent object, allowing offsets such as positioning the camera slightly behind the player

1.11.3 CPU vs GPU

The scene graph is a CPU-side abstraction whose purpose is to help organise and manage complex scenes.

The CPU:

- stores the scene graph structure
- traverses the hierarchy

- combines transformation matrices to compute the final world transformation for each object M_{world}
- sends the final transformation matrix to the GPU

The GPU:

- receives the final transformation matrices
- knows nothing about the scene graph hierarchy
- performs parallel vertex transformations ($M_{\text{world}} \times P$)

1.11.4 Three.js example

```
// Each mesh is a node
const sun = new THREE.Mesh(sphereGeom, sunMaterial);
const earth = new THREE.Mesh(sphereGeom, earthMaterial);
const moon = new THREE.Mesh(sphereGeom, moonMaterial);

// Build the scene graph
scene.add(sun); // Sun is a child of the root scene
sun.add(earth); // Earth is a child of the Sun
earth.add(moon); // Moon is a child of the Earth

// Set local transformations
earth.position.x = 10; // Position Earth relative to the Sun
moon.position.x = 2; // Position Moon relative to the Earth

// Animation loop
function animate(){

    // Local transformations
    sun.rotation.y += 0.005; // Sun spins on its axis
    earth.rotation.y += 0.01; // Earth spins on its axis

    // Because Earth is offset from the Sun,
    // rotating the Sun causes Earth to orbit it
    // Likewise, the Moon inherits Earth's movement

    // Render automatically traverses the scene graph
    // and computes world matrices
    renderer.render(scene, camera);
}
```

1.12 Ray tracing

Ray tracing is a rendering technique that simulates the physical behaviour of light by tracing the paths that rays of light take through a scene.

It works backwards:

- rays are cast from the camera through each pixel
- the renderer finds the closest object each ray intersects
- lighting is computed at the intersection point
- lighting is therefore only computed surfaces that are actually visible
- additional rays may be spawned to simulate further light interactions such as shadows, reflections, and refractions

1.12.1 Rasterisation vs Ray Tracing

Rasterisation is excellent at determining visibility (which polygons are visible), but it does not naturally simulate light transport. This is because:

- rasterisation processes one triangle at a time
- each triangle is shaded largely independently
- when drawing a triangle, the GPU does not know about the existence of any other triangle
- therefore, it cannot calculate how one triangle may affect the lighting of another triangle (such as blocking light from reaching the triangle, or reflecting light onto it)

Ray tracing, by contrast, is aware of the entire scene and can simulate the paths taken by light rays, producing realistic shadows, reflections, refractions, and global illumination.

1.12.2 Ray equation

A ray is a half-line defined by its starting point and its direction:

$$P(t) = O + t\vec{D}$$

where:

- $P(t)$ is the 3D point located a distance t along the ray
- O is the origin of the ray
- $\vec{D}$ is the normalised direction vector
- t is a scalar parameter representing distance along the ray with $t \geq 0$

This equation is used to determine intersections between light rays and objects in the scene.

1.12.3 Final colouring

At each ray–object intersection, the ray tracer computes the final colour by summing several light contributions:

- local illumination: direct light from light sources, including diffuse and specular components. Shadow rays are used to determine whether each light source is visible from the intersection point.
- reflected illumination: light arriving from elsewhere in the scene, gathered by the recursive reflection ray